Machine Translated by Google

Hybrid Chatovacia P2P network

Data transmission, computer networks and protocols (PDS)

Brno University of Technology

Bc. Michal Ormoš

April 28, 2019

Machine Translated by Google

Contents

1 Assignment 2
2 Implementation 2
Peer Implementation . . . 2.1 2.2 2
Implementation of node 3
RPC implementation 3
2.3 3 Communication 4
4 Testing the project 4
Bc. Testing with Bc. Vojtÿch Matÿjíÿek (xmatej51) . 4.1 Testing with 4
ÿuboš Mjachky (xmjach00) . 4.2 1 Sequence diagram 5
. 6

Machine Translated by Google

1 Assignment

The goal of this project was to create a hybrid client-client [2] chat application. This application was to consist of a chat client (also hereinafter referred to as peer), a registration node (also hereinafter referred to as node) and an RPC module, for testing purposes. The task of the registration node is to create a space to which chat clients can connect. In this way, the node collects information about available clients and can provide them with this client database at the client's request. Thanks to this information, the client itself finds out the names and addresses of clients in its vicinity and can connect to them directly without the help of a registration node and send them messages. As you can already see, the whole point of a client-client chat application is to leave the server out of the actual process of sending messages between two clients and thus ensure (even if this application does not provide encryption of messages) communication between two people, without it being processed by a third party.

2 Implementation

The Python language was chosen as the implementation language due to the ease of working with the JSON format, which this project will require in large quantities. Despite the effort to implement a full-mesh network, I was unable to create the cooperation of nodes (full-mesh network), therefore the registration nodes in this project cannot communicate with each other or exchange information.

2.1 Peer implementation

The role of a peer in a project is three basic things.

- Connecting to the registration node
- Connecting to the RPC module
- Listening on the management port

The client is implemented using a single source file that contains the Client class. During initialization, it creates a UDP socket [1] for receiving chat messages according to the input arguments and reserves the given socket so that the operating system does not change it.
On this socket, it waits for MESSAGE messages from other clients and, if received, sends an ACK message to the address that sent the message.

Next, according to the input arguments, the client creates a thread on which it starts sending HELLO messages periodically every 10 seconds to its registration node, which was assigned to it. Since ACK is not checked for HELLO messages, the client sends messages to the registration node in a deaf manner. The client remembers at what time it sent the last HELLO message, and using periodic time detection in the timestamp format, it finds out when 10 seconds have passed since the last message was sent, and thus it has to send a new one and update its time. In conjunction with

Machine Translated by Google

with its registration node, the client can ask the node for a list of other clients in this network upon request from the RPC and then confirm that it has received it. Thanks to which it can send messages to other clients in the network without the participation of the node in this communication, i.e. send the message directly.

In the second thread, the client creates another UDP socket for receiving messages from RPC. And it does so by assigning a port to its chat IPv4 address as follows, taking its id that was assigned to it at the input and adding nines to it until its port length is four. That is, if the ID number is 7, its RPC port will be 7999. On this RPC, it listens on a classic UDP socket for messages from its RPC authority, which can ask it to get a list of other clients from its node, contact another client, or disconnect from the current node and connect to another node.

2.2 Node implementation

The implementation of the registration node is similar to the implementation of the client. When the node starts, it creates a UDP socket on which, according to the input arguments, it waits for messages from clients.
News can come in three forms.

- HELLO - The client reports to his node, where he sends information about himself, which the registration node stores in its internal database, which it subsequently provides when requested. The node checks whether the given client already exists in its database, and if so, it just updates the time stamp of the last time it logged in. Because if the client does not log in using the HELLO management for more than 30 seconds, it is kicked out of the registration node.

- GETLIST - The client can request the registration node for the database of clients connected to it. The node complies by sending a LIST message and waiting for confirmation that the client received it with an ACK.

- ACK - The client announces that it has successfully received the last transaction.

In a special thread, it creates an RPC socket on which it listens for RPC commands, and determines the RPC port in the same way as the client did. The latter can command it to display the current database of its nodes.

2.3 RPC Implementation

RPC is implemented as another client whose role is to send special messages to the client or node. The implementation was done using the equivalent of a UDP socket, but there was a problem with how the RPC module would find out where to contact the node or the client. Each time a client or node is started, the given client/node is written into a special network.dat file, where it creates a record of itself, namely type (peer/node), id, ip_address and special RPC_port. Thanks to this, RPC has an overview of where and how individual clients are running

Machine Translated by Google

nodes. If the client or node is terminated (by the user or by the system), it deletes its record from the given file. If this deleted record is the last one, the entire file will be deleted. The results of RPC commands are written to the respective node/peer and not to the RPC output itself.

3 Communication

Communication takes place by sending JSON messages that are encoded. The yabencode library was used to encode them. I found the implementation of the bank coding module unnecessary. Because I came across a few libraries that do it perfectly and their content is free to distribute. Therefore, this library was added to the program as a module, since the test virtual did not contain it. It is also the only external code that was taken over in the program.[1]

Except for the HELLO and ERROR messages, each communication message is acknowledged with an ACK.

4 Project testing

Since I was unable to implement a full-mesh network of nodes stably under any circumstances, I left the application testing to the very end, which was a mistake, and all my known colleagues already had four people with whom they tested, so I am attaching my slightly more modest testing with two people. Currently implemented RPC commands:

- –peer –command message –from <username1> –to <username2> –message <message>
- –peer –command getlist
- –peer –command peers
- –peer –command reconnect –reg-ipv4 <IP> –reg-port <port>
- –node –command database

4.1 Testing with Bc. Vojtÿch Matejíÿek (xmatej51)

Implementation language: Python

Tested elements

- Testing took place at the public address of the Merlin server and a local PC
- RPC calls for peer/node combinations
- Disconnecting peers from nodes

1https://pypi.org/project/yabencode/

Machine Translated by Google

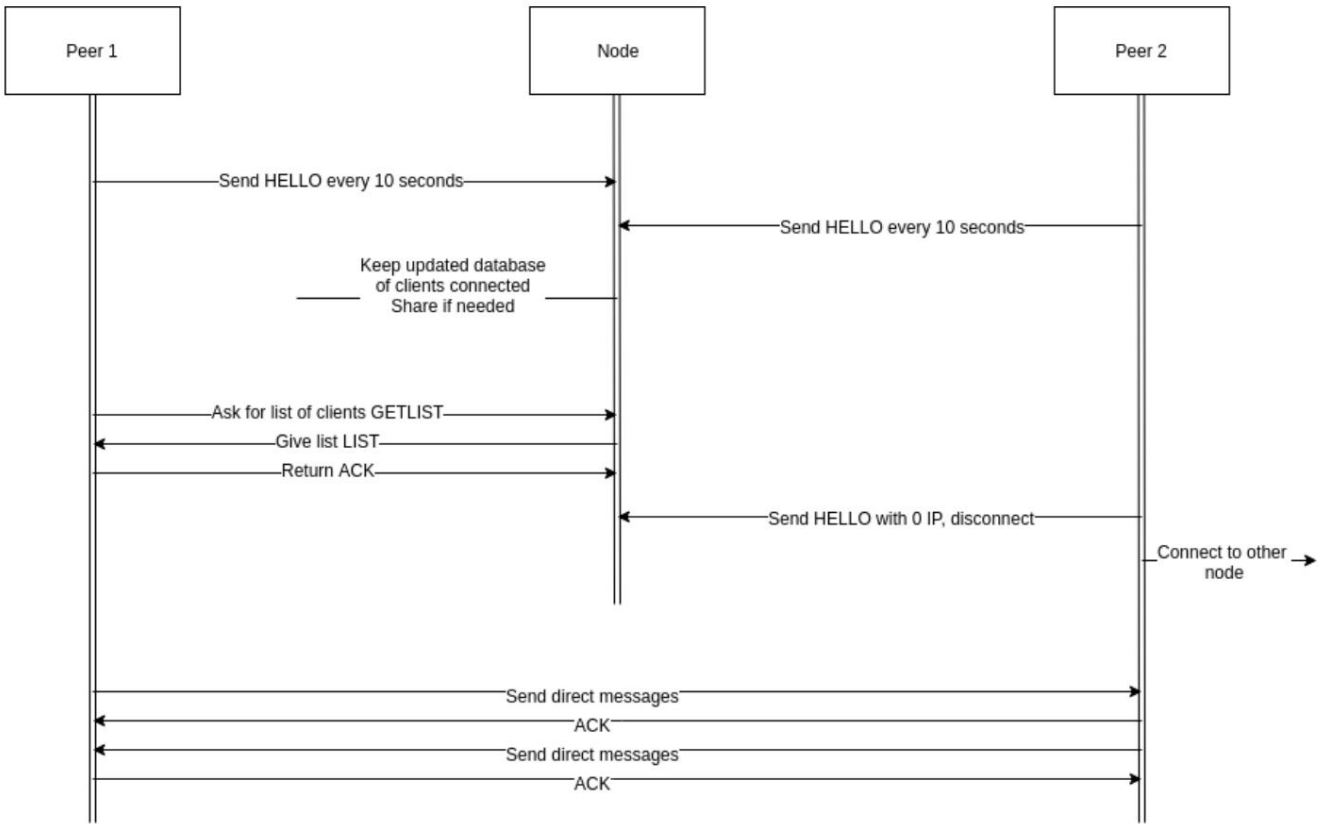
4.2 Testing with Bc. ÿuboš Mjachky (xmjach00)

- Implementation language: Python
- Tested elements
  - – Connecting/disconnecting peers from mutual storage
  - – Sending peer-to-peer management to each other through different nodes
- Implementation details:
  - – Used one thread and one request (message). N threads for n requests.
  - – Testing took place on the public address of the Merlin server
  - – The entire project was implemented by xmjach00
  - – RPC listens on a separate thread and handles requests, result/error
    The message will be printed to the RPC output.
  - – RPC communication takes place on the basis of the AF_UNIx socket, which is reliable in terms of delivery
  - – a simple timer is used to track the time of nodes or clients

Machine Translated by Google

1 Sequence diagram

fig. 1: Program sequence diagram



Machine Translated by Google

Literature

[1] Python: Wiki, https://
wiki.python.org/moin/UdpCommunication

[2] KOEGEL BUFORD, John F, Hong Heather YU and Eng Keong LUA. P2P networking and applications. Burlington: Morgan Kaufmann, 2009, xxi, 415 pp. : illus. ISBN 978-0-12-374214-8.